Normal forms for Context-Free Grammars

Normal forms

- Every context-free grammar that does not generate the empty string can be transformed into an equivalent one in Chomsky normal form or Greibach normal form. "Equivalent" here means that the two grammars generate the same language.

- Because of the especially simple form of production rules in Chomsky Normal Form grammars, this normal form has both theoretical and practical implications.

- For instance, given a context-free grammar, one can use the Chomsky Normal Form to construct a polynomial-time algorithm which decides whether a given string is in the language represented by that grammar or not (the CYK algorithm).

Properties of context-free languages

- An alternative and equivalent definition of context-free languages employs non-deterministic push-down automata: a language is context-free if and only if it can be accepted by such an automaton.
- A language can also be modeled as a set of all sequences of terminals which are accepted by the grammar. This model is helpful in understanding set operations on languages.
- The union and concatenation of two context-free languages is context-free, but the intersection need not be.
- The reverse of a context-free language is context-free, but the complement need not be.

Properties of context-free languages

- Every regular language is context-free because it can be described by a regular grammar.
- The intersection of a context-free language and a regular language is always context-free.
- There exist context-sensitive languages which are not context-free.
- To prove that a given language is not context-free, one may employ the pumping lemma for context-free languages.
- The problem of determining if a context-sensitive grammar describes a context-free language is undecidable.

Normal forms for Context-Free Grammars

The goal is to show that every CFL (without ε) is generated by a CFG in which all productions are of the form A □ BC or A □ a, where A, B, C are variables, and a is a terminal.

Normal forms for Context-Free Grammars

- A number of simplifications is inevitable:

1. The elimination of useless symbols, “variables or terminals that do not appear in any derivation of a terminal string from the start symbol”.
2. The elimination of ε-productions, those of the form $A \rightarrow \varepsilon$ for some variable A.
3. The elimination of unit productions, those of the form $A \rightarrow B$ for variables A and B.

Eliminating useless symbols

- A symbol X is useful for Grammar G = {V, T, P, S}, if there is some derivation of the form S =>* a X b =>* w , where w ϵ T*
- X ϵ V or X ϵ T
- The sentential form of a X b might be the first or last derivation
- If X is not useful, then X is useless

Eliminating useless symbols

- Characteristics of useful symbols (for instance X):

1. X is generating if X =>* w for some terminal string w. Every terminal is generating since w can be that terminal itself, which is derived by 0 steps.

2. X is reachable if there is a derivation
   S =>* a X b for some a and b.

   A symbol which is useful is surely to be both generating and reachable.

Eliminating useless symbols

Eliminating the symbols which are not generating first followed by eliminating the symbols which are not reachable from the remaining grammar, this will generate a grammar consisting of only useful symbols.

Eliminating useless symbols

- Example 7.1
- If we have the following grammar:

$$S \rightarrow AB \mid a$$
$$A \rightarrow b$$

Eliminating useless symbols

- Example 7.1
- Notice that a and b generate themselves "terminals", S generates a, and A generates b. B is not generating.
- After eliminating B:

$$S \to a$$
$$A \to b$$

Eliminating useless symbols

- Example 7.1
- Notice that only S and a are reachable after eliminating the non-generating B.
- A is not reachable; so it should be eliminated.
- The result :

$$S \rightarrow a$$

- This production itself is a grammar that has the same result, which is {a}, as the original grammar.

Computing the generating and reachable symbols

- Basis: Every Symbol of T is obviously generating; it generates itself.
- Induction: If we have a production $A \rightarrow a$, and every symbol of a is already known to be generating, then A is generating; because it generates all and only generating symbols, even if $a = \varepsilon$; since all variables that have ε as a production body are generating.
- Theorem: The previous algorithm finds all and only the Generating symbols of G

Computing the generating and reachable symbols

- Basis : For a grammar $G = \{V, T, P, S\}$
  S is surely reachable.

- Induction: If we discovered that some variable A is reachable, then for all productions with A in the head (first part of the expression), all the symbols of the bodies (second part of the expression) of those productions are also reachable.

- Theorem: The above algorithm finds all and only the Reachable symbols of G

Eliminating useless symbols

- So far, the first step, which is the elimination of useless symbols is concluded.
- Now, for the second part, which is the elimination of ε-productions.

Eliminating ε-productions

- The strategy is to have the following:
  if L is CFG, then L – {ε} is also CFG
- This is done through discovering the nullable variables. A variable for instance A, is nullable if: A =>* ε .
- Whenever A appears in a production body, A might or might not derive ε

Eliminating ε-productions

- Basis: If $A \rightarrow \varepsilon$ is a production of G, then A is nullable
- Induction: If there is a production $B \rightarrow C_1 C_2 \ldots C_k$ such that each C is a variable and each C is nullable, then B is nullable

Eliminating ε-productions

- Theorem: For any grammar G, the only nullable symbols are the variables that derive ε in previous algorithm
- Proof:

  <u>for one step</u> : A □ ε must be a production, then this implies that A is discovered as nullable (as in basis).

  <u>for N > 1 steps:</u> the first step is A □ $C_1 C_2 \ldots C_k$ □ ε , each C_i derives ε by a sequence < N steps.

  By the induction, each C_i is discovered by the algorithm to be nullable. So by the inductive step, A is eventually found to be nullable.

Eliminating ε-productions

- If a grammar G_1 is constructed by the elimination of ε-productions " using the previous method " of grammar G, then

$$\mathbf{L(G_1) = L(G) - \{\varepsilon\}}$$

Eliminating unit productions

- The last part concerns the eliminating of unit productions
- Any production of the form A □ B , where A and B are variables, is called a unit production.
- These production introduce extra steps in the derivations that obviously are not needed in there.

Eliminating unit productions

- Basis: (A, A) is a unit pair of any variable A, if
  A ==>* A by 0 steps.

- Induction: Let's (A, B) be a unit pair, and let B □ C is a production, where A, B, and C are variables, then we can conclude that
  (A, C) is also a unit pair.

- Theorem: The previous algorithm (basis and induction) finds exactly all the unit pairs for any grammar G.

Eliminating unit productions

- Example 7.12

$$
\begin{array}{lcl}
I & \rightarrow & a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
F & \rightarrow & I \mid (E) \\
T & \rightarrow & F \mid T * F \\
E & \rightarrow & T \mid E + T
\end{array}
$$

Eliminating unit productions

- Example 7.12

Pair	Productions
(E, E)	$E \to E + T$
(E, T)	$E \to T * F$
(E, F)	$E \to (E)$
(E, I)	$E \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(T, T)	$T \to T * F$
(T, F)	$T \to (E)$
(T, I)	$T \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(F, F)	$F \to (E)$
(F, I)	$F \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$
(I, I)	$I \to a \mid b \mid Ia \mid Ib \mid I0 \mid I1$

Eliminating unit productions

- Example 7.12

  After eliminating the unit productions, the generated grammar is:

$$
\begin{aligned}
E &\rightarrow E+T \mid T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
T &\rightarrow T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
F &\rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1
\end{aligned}
$$

  This grammar has no unit productions and still generates the same expressions as the previous one.

Chomsky Normal Form

Conclusion of all three elimination stages:

Theorem: If G is a CFG which generates a language that consists of at least one string along with ε, then there is another CFG G_1 such that:

$L\{G_1\} = L\{G\} - \{\varepsilon\}$, "no ε-productions", and G_1 has neither unit productions nor useless symbols

Chomsky Normal Form

- Proof: Start by performing the elimination of ε-productions. Then perform the elimination of unit productions, so the resulting grammar won't introduce any ε-productions since the new bodies are still identical to some bodies of the old grammar. Finally, perform the elimination of useless symbols, and since this eliminates productions and symbols, it will never reintroduce any ε-productions nor unit productions

Chomsky Normal Form

Every nonempty CFL without ε has grammar G in which all productions are in one of the following forms:

- A □ BC , where A, B, and C are variables or
- A □ a , where A is a variable and a is a terminal

- Also G doesn't contain any useless symbols

- A grammar complying to these forms is called a Chomsky Normal Form (CNF).

Chomsky Normal Form

- The construction of CNF is performed through:

1. Arrangement of all bodies of length 2 or more to contain only variables.

2. Breaking bodies of length 3 or more into a cascade productions, where each one has a body consisting of 2 variables.

Chomsky Normal Form

- Example 7.15

$$
\begin{aligned}
I &\rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1 \\
F &\rightarrow I \mid (E) \\
T &\rightarrow F \mid T * F \\
E &\rightarrow T \mid E + T
\end{aligned}
$$

Chomsky Normal Form

- Example 7.15
- First: we introduce new variables to represent terminals:

$$A \to a \quad B \to b \quad Z \to 0 \quad O \to 1$$
$$P \to + \quad M \to * \quad L \to (\quad R \to)$$

Chomsky Normal Form

- Example 7.15
- Second: We make all bodies either a single terminal or multiple variables:

$$
\begin{array}{lcl}
E & \rightarrow & EPT \mid TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
T & \rightarrow & TMF \mid LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
F & \rightarrow & LER \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
I & \rightarrow & a \mid b \mid IA \mid IB \mid IZ \mid IO \\
A & \rightarrow & a \\
B & \rightarrow & b \\
Z & \rightarrow & 0 \\
O & \rightarrow & 1 \\
P & \rightarrow & + \\
M & \rightarrow & * \\
L & \rightarrow & (\\
R & \rightarrow &)
\end{array}
$$

Chomsky Normal Form

- Example 7.15
- Last step: we make all bodies either a single terminal or two variables:

$$
\begin{array}{lcl}
E & \rightarrow & EC_1 \mid TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
T & \rightarrow & TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
F & \rightarrow & LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO \\
I & \rightarrow & a \mid b \mid IA \mid IB \mid IZ \mid IO \\
A & \rightarrow & a \\
B & \rightarrow & b \\
Z & \rightarrow & 0 \\
O & \rightarrow & 1 \\
P & \rightarrow & + \\
M & \rightarrow & * \\
L & \rightarrow & (\\
R & \rightarrow &) \\
C_1 & \rightarrow & PT \\
C_2 & \rightarrow & MF \\
C_3 & \rightarrow & ER
\end{array}
$$